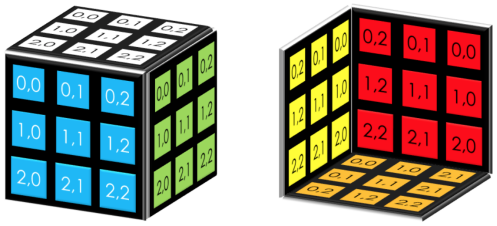
Artificial Intelligence

Lab 1

The objective of this lab is to implement a program capable of solving a Rubik's Cube. You will need to complete this code in order to implement the "solve" button.

1 Class Cube

The `Cube` class models the game. For each face (top, bottom, right, front, left, back), a matrix of characters indicates the color of each sticker on the face. The possible characters are 'W' (white), 'R' (red), 'G' (green), 'B' (blue), 'Y' (yellow), and 'O' (orange). The following figure illustrates the row and column indices for the different faces:



The functions `turnUp`, `turnLeft`, `turnFront`, `turnRight`, `turnDown`, and `turnBack` perform clockwise rotations of the corresponding faces. The functions `returnUp`, `returnLeft`, `returnFront`, `returnRight`, `returnDown`, and `returnBack` perform counterclockwise rotations.

The function `isSolved` returns true only if the cube configuration matches the solved state described above, where each face has a uniform color (white for the top face, etc.).

2 Class RubiksCubeSimulator

This class handles the graphical user interface that displays the cube. It contains an object of type `Cube`, which represents the cube shown in the interface. It also

contains the `main` function that creates the window. The `actionPerformed` function is triggered each time the user presses a button in the interface. The solving process will be launched in this function when the user presses the solve button.

3 Class Astar

The `Astar` class is intended to implement the A^* algorithm to solve the Rubik's Cube. It contains an object of type `State`, which is the root of the search tree built by the algorithm. The `State` class is described in the next section.

The class also contains an attribute `explored`, of type `StateSet`, which stores the set of nodes (states of type `State`) that have already been expanded. You can interact with this attribute using the following methods:

- `add`: adds a state to the explored set,
- `contains`: returns true if the given state is already in the explored set.

Finally, the class contains an attribute `frontier`, of type `PriorityQueueState`, which represents the frontier. You can interact with it using the following methods:

- `push`: adds a state to the frontier,
- `pop`: returns the state with the smallest value (number of actions + heuristic).

Note that the `push` function automatically checks whether a state (cube configuration) is already in the frontier, and if so, keeps only the better one.

The constructor of the `Astar` class initializes these data structures. It takes as input the initial cube configuration, which becomes the root of the search tree.

The `solve` function implements the A^* algorithm starting from the given cube. This is the function you must implement. It returns a list of strings representing the sequence of actions required to reach the goal state.

4 Class State

The `State` class represents a node in the search tree. Each state corresponds to a cube configuration represented by an object of type `Cube`.

The attribute `nbrActions` stores the number of actions taken from the root to reach this state.

The attribute `pere` represents the parent state that generated this state. It is used to reconstruct the solution path once the goal state is found.

The attribute `actionPere` indicates the action used to generate this state from its parent. It is also essential for reconstructing the solution. The possible action identifiers are:

0	turnUp	U
1	turnLeft	L
2	turnFront	F
3	turnRight	R
4	turnDown	D
5	turnBack	B
6	returnUp	U'
7	returnLeft	L'
8	returnFront	F'
9	returnRight	R'
10	returnDown	D'
11	returnBack	B'

The attribute `valH` stores the heuristic value of the state, computed when the state is created.

The constructor takes as input: - the cube configuration, - the number of actions, - the parent state, - and the action identifier (an integer between 0 and 11).

It also computes the heuristic value and stores it in `valH`.

The function `expand` generates all child states by applying the twelve possible rotations to the current cube configuration. The children are stored in a `LinkedList` and returned.

The classes implementing the `Heuristic` interface (`HeuristicLabel` and `HeuristicCube`) compute heuristic values. The method `value` computes the heuristic for a given state.

These heuristics are based on relaxations of the Rubik's Cube problem: - `HeuristicLabel`: assumes stickers move independently, - `HeuristicCube`: assumes cube pieces (with 2 or 3 stickers) move independently.

5 Work to be done

You must implement the following classes and functions.

5.1 Function solve in class `Astar`

Implement the `solve` function so that it applies the A^* algorithm to the Rubik's Cube problem.

Initially, you will implement a uniform-cost search (i.e., ignore the heuristic). After implementing it, test your algorithm using the graphical interface (especially by scrambling the cube before solving).

Determine from how many random moves the algorithm can no longer find a solution efficiently (too slow or runs out of memory).

5.2 Class HeuristicLabel

Now include an admissible heuristic to guide the search.

Start by changing the heuristic used in the `State` class to `HeuristicCube` (already implemented). Test whether A^* can solve more scrambled cubes than uniform-cost search.

Then implement the `value` function of `HeuristicLabel` yourself.

This heuristic is based on relaxing the problem by assuming that each sticker can move independently. In other words, you can move one sticker without affecting others.

The heuristic computes the 3D Manhattan distance of each sticker and counts the cost required to move each sticker to its correct face.

The heuristic value is: - the sum of all 3D Manhattan distances, - divided by the number of stickers that can change face in a single rotation.

Implement this function and test it via the interface (after switching the heuristic in `State`). Evaluate how scrambled a cube can be while still being solvable using A^* with your heuristic.